

RESPUESTAS A LAS PREGUNTAS TEÓRICAS SOBRE FUNCIONES EN PSEINT

PREGUNTA 1: ¿Es posible el uso de funciones anidadas?

RESPUESTA: NO, PSeInt NO soporta funciones anidadas (definir una función dentro de otra).

JUSTIFICACIÓN: Las funciones en PSeInt deben declararse al mismo nivel, fuera del algoritmo principal. No se puede definir una función dentro de otra función ni dentro del algoritmo principal.

Ejemplo INCORRECTO:

```
Funcion Externa()  
    Funcion Interna() // ¡ESTO NO FUNCIONA EN PSEINT!  
        Escribir "Esto dará error"  
    FinFuncion  
FinFuncion
```

Forma CORRECTA: Todas las funciones deben estar al mismo nivel jerárquico, declaradas antes del algoritmo principal.

PREGUNTA 2: ¿Es posible que una función acceda a un array situado en el código principal?

RESPUESTA: SÍ, pero SOLO si se pasa como parámetro POR REFERENCIA.

JUSTIFICACIÓN: En PSeInt no existen variables globales accesibles directamente desde las funciones. Sin embargo, los arrays se pasan automáticamente por referencia cuando se envían como parámetros, lo que permite que la función acceda y modifique el array original del código principal.

Ejemplo:

```
Algoritmo Principal  
    Dimension datos[5]  
    Para i <- 1 Hasta 5 Hacer  
        datos[i] <- i  
    FinPara  
    ModificarArray(datos, 5)  
    // El array "datos" estará modificado después de llamar a la función  
FinAlgoritmo  
  
Funcion ModificarArray(arr, n)  
    Para i <- 1 Hasta n Hacer  
        arr[i] <- arr[i] * 10 // Modifica el array original  
    FinPara  
FinFuncion
```

PREGUNTA 3: ¿Es posible que una función devuelva un array al código principal?

RESPUESTA: NO directamente. PSeInt NO permite que una función retorne un array como valor de retorno.

JUSTIFICACIÓN: Las funciones en PSeInt solo pueden retornar valores simples (enteros, reales, lógicos, cadenas), no estructuras de datos complejas como arrays.

SOLUCIÓN: La forma correcta es pasar el array como parámetro (por referencia) y modificarlo dentro de la función. Como los arrays se pasan por referencia, los cambios realizados dentro de la función afectan al array original.

Ejemplo INCORRECTO:

```
Funcion resultado <- CrearArray() // NO FUNCIONA
  Dimension resultado[10]
  Para i <- 1 Hasta 10 Hacer
    resultado[i] <- i * 2
  FinPara
FinFuncion
```

Ejemplo CORRECTO:

```
Funcion LlenarArray(arr, n)
  Para i <- 1 Hasta n Hacer
    arr[i] <- i * 2
  FinPara
FinFuncion

Algoritmo Principal
  Dimension miArray[10]
  LlenarArray(miArray, 10) // El array se modifica directamente
  // Ahora miArray contiene los valores modificados
FinAlgoritmo
```

PREGUNTA 4: Corrección del código proporcionado

CÓDIGO ORIGINAL (CON ERRORES):

```
Funcion LlamarAMiFuncion(N, MiArray)
  Escribir "Array modificado en el programa principal:"
  Para i <- 1 Hasta N
    Escribir MiArray[i]
  FinPara
FinFuncion

Algoritmo array
  Dimension MiArray[10]
  Escribir "Ingrese el tamaño del array <10: "
  Leer N
  LlamarAMiFuncion(N, MiArray[])
  Para i <- 1 Hasta N
    Dimension Array[10]
    Array[i] <- i * 2
  FinPara
FinAlgoritmo
```

ERRORES IDENTIFICADOS:

1. **Error de concepto:** La función "LlamarAMiFuncion" solo muestra el array, pero no lo modifica. La modificación se hace después en el algoritmo principal.
2. **Sintaxis incorrecta:** Se pasa "MiArray[]" con corchetes, cuando debe ser solo "MiArray".
3. **Declaración dentro del bucle:** Se declara "Dimension Array[10]" DENTRO del bucle Para, lo cual es ineficiente y genera errores. Las dimensiones deben declararse una sola vez.
4. **Variables no definidas:** La variable "N" no está declarada con su tipo.
5. **Lógica invertida:** La modificación del array se hace en el algoritmo principal cuando debería hacerse en la función.
6. **Sin validación:** No valida que N sea menor o igual a 10.
7. **Nombre confuso:** Se usa "Array" como nombre de variable, lo cual puede generar confusión.

CÓDIGO CORREGIDO:

```
// Función que modifica el array
Funcion ModificarArray(MiArray, N)
  Para i <- 1 Hasta N Hacer
    MiArray[i] <- i * 2
  FinPara
FinFuncion

Algoritmo ArrayCorregido
  Definir N, i Como Entero
  Dimension MiArray[10]

  Escribir "Ingrese el tamaño del array (<=10): "
  Leer N

  // Validar entrada
  Si N > 10 O N < 1 Entonces
    Escribir "Error: El tamaño debe estar entre 1 y 10"
  SiNo
    // Llamar a la función que modifica el array
    ModificarArray(MiArray, N)

    // Mostrar el array modificado
    Escribir "Array modificado:"
    Para i <- 1 Hasta N Hacer
      Escribir "Posición ", i, ": ", MiArray[i]
    FinPara
  FinSi
FinAlgoritmo
```

MEJORAS APLICADAS:

- ✓ **Función específica:** Ahora la función tiene un propósito claro (modificar el array)
 - ✓ **Paso correcto de parámetros:** Se pasa "MiArray" sin corchetes
 - ✓ **Dimensión correcta:** Se declara una sola vez fuera del bucle
 - ✓ **Validación de entrada:** Se verifica que N esté entre 1 y 10
 - ✓ **Variables definidas:** Todas las variables tienen su tipo declarado
 - ✓ **Código legible:** Mejor estructura y nombres descriptivos
 - ✓ **Separación de responsabilidades:** La función modifica, el algoritmo muestra
-

CONCLUSIONES:

1. PSeInt no permite funciones anidadas, todas deben declararse al mismo nivel.
2. Los arrays se pueden pasar a funciones por referencia, permitiendo su modificación.
3. Las funciones no pueden retornar arrays directamente, pero sí modificarlos por referencia.
4. Es fundamental validar las entradas y declarar correctamente las dimensiones de los arrays.
5. El código debe ser claro, con funciones que tengan un propósito específico y bien definido.